Correctness, Measurement-Bias, and Numerical-Accuracy Auditing of a Dense Matrix Multiplication and Strassen Implementation

SAHITH KANCHARLA, Virginia Tech, Department of Computer Science, USA

A prior, unpublished AVX-512 dense-matrix-multiplication (DGEMM) and Strassen implementation is audited for three questions: whether it does what it claims, whether its performance wins reflect real hardware effects rather than measurement artifacts, and whether its numerical-accuracy discussion can be extended with a current literature result not yet benchmarked anywhere. A correctness audit finds and fixes four real, execution-verified bugs, the most serious a packing error that silently computes the product in the wrong order for non-square shapes. A large, roughly two-times OpenMP dynamic-scheduling performance win, first measured under virtualization, is shown through controlled, repeated measurement – including the heterogeneous performance/efficiency-core case scheduling theory favors most – not to reproduce; only core placement survives repetition, extending a general measurement-bias methodology into a new domain. A 2024 accuracy-optimized alternate-basis Strassen variant is implemented as a real recursive block algorithm, its formula symbolically verified against the source paper's straight-line program first, catching a transcription error. Measured under realistic packed, blocked, multi-threaded execution for what appears to be the first time, it shows consistently lower error than classical Strassen at matched recursion depth, at a measurable throughput cost the source paper does not report. All results are backed by a public, reproducible harness and dataset.

CCS Concepts: • **Mathematics of computing** → **Mathematical software**; • **Computing methodologies** → **Shared memory algorithms**; • **Software and its engineering** → *Software performance*; • **General and reference** → *Empirical studies*.

Additional Key Words and Phrases: dense matrix multiplication, Strassen's algorithm, measurement bias, reproducibility, OpenMP scheduling, numerical accuracy, correctness testing

ACM Reference Format:
Sahith Kancharla. 2026. Correctness, Measurement-Bias, and Numerical-Accuracy Auditing of a Dense Matrix Multiplication and Strassen Implementation. *ACM Trans. Math. Softw.* 0, 0, Article 0 (January 2026), 19 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

1.1 Motivation

High-performance DGEMM and fast-matrix-multiplication engineering has a deep, well-cited lineage: Goto and van de Geijn's multilevel blocking and packing model [13], generalized into the portable BLIS framework [19], and Huang et al.'s demonstration that Strassen's extra additions can be fused into an already-optimized DGEMM's packing routines at essentially no extra workspace cost [15]. Adversarial re-execution – taking a specific implementation, running it, and checking

No external funding supported this work. No part of this paper has been presented at a conference or published in a newsletter.
Author's address: Sahith Kancharla, Virginia Tech, Department of Computer Science, USA, sahith@vt.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
0098-3500/2026/1-ART0 $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

whether it actually does what its own writeup claims – is a useful check regardless of who wrote the original code, including the same author. This project started as exactly that exercise, applied to the author's own earlier, unpublished AVX-512 DGEMM/Strassen implementation (an undergraduate course project, never submitted or peer-reviewed anywhere) against its own accompanying writeup – and the audit surfaced real bugs (Section 4), which is itself evidence that this kind of check is worth doing routinely, not just once and not just by someone other than the original author.

The audit also surfaced a methodological question with broader implications than this one codebase: a large, apparently clean OpenMP scheduling performance win, measured once on a virtualized development environment, turned out not to survive a properly repeated measurement (Section 5). This is a domain-specific instance of a general phenomenon Mytkowicz et al. documented in 2009 – that "insignificant" experimental-setup choices can flip a benchmark's conclusion [17] – and, as far as our own literature search found (Section 2), it has not previously been demonstrated as a targeted case study for a DGEMM/OpenMP scheduling claim specifically. Given how routinely such benchmarks are now run inside cloud VMs and WSL2-style virtualized dev environments, that gap seemed worth closing with a concrete, repeatable example rather than a general warning.

Finally, in the course of extending this project's own numerical- accuracy work, we found a genuinely new (2024) result in the fast- matrix-multiplication literature – an alternate-basis Strassen variant with a provably better accuracy bound [11] – that, as far as we could determine, had never been implemented as a real recursive block algorithm or benchmarked on hardware under realistic packed/blocked execution; the source paper reports only asymptotic operation counts and an accuracy-vs-size plot, no wall-clock or phase-timed performance data at all. Section 6 closes that gap and reports the accuracy/performance trade-off its own authors did not measure.

1.2 Contributions

(1) **A correctness audit of an earlier, unpublished AVX-512 DGEMM/Strassen implementation**, finding and fixing four real bugs with execution evidence (not just static reading), the most significant a packing bug that silently computed $B \cdot A$ instead of $A \cdot B$; a second-pass, repo-wide sweep found and fixed six further bugs, including a regression introduced by this audit's own first-pass fix. See Section 4.

(2) **A measurement-bias case study**: a large (2×-class) OpenMP dynamic-scheduling performance win does not reproduce under proper repeated measurement – including in the heterogeneous performance/efficiency-core case theory predicts should favor it most; only raw core placement survives repetition. See Section 5.

(3) **Dumas et al.'s 2024 alternate-basis Strassen, implemented and measured**: a real recursive block algorithm built on infrastructure identical to a classical-Strassen baseline, symbolically verified before implementation (catching a real subscript-convention transcription bug), then measured for both accuracy (confirms the paper's claim, under conditions the paper itself never tested) and performance cost (a number nobody has published). See Section 6.

(4) Supporting material: a from-scratch AVX2 DGEMM tuned against production BLAS, hardware-counter root-cause analysis explaining *why* several classical performance results hold, cross- compiler codegen differences, a from-scratch classical-Strassen rebuild with a measured optimized-vs-optimized crossover, and a rectangular/irregular-shape performance-cliff finding. See Section 7.

(5) A public, reproducible harness and dataset backing every number in this paper. See Section 9.

2 RELATED WORK

2.1 High-performance GEMM engineering

GEMM is the Level-3 BLAS routine this entire paper studies; Dongarra, Du Croz, Duff and Hammarling's original Level-3 BLAS specification [10] is the standard it and every library we benchmark against (OpenBLAS, BLIS, MKL) implement. Goto and van de Geijn's "Anatomy of High-Performance Matrix Multiplication" [13] establishes the multilevel cache-blocking and operand-packing model (an $M_C \times K_C$ panel of A and a $K_C \times N_C$ panel of B, packed into contiguous buffers and driven through a small register-blocked microkernel) that underlies essentially all modern high-performance DGEMM, including both the implementation we audit and our own from-scratch AVX2 microkernel work (Section 7). Van Zee and van de Geijn's BLIS framework [19] generalizes that model so that only a small, architecture-specific microkernel needs re-tuning per target – a design pattern our own portable, runtime-configurable blocking layer (FINDINGS.md Part B.12) independently converges on at smaller scale. These are the ceiling our own plain-DGEMM measurements are stated relative to, not a target we claim to beat.

2.2 Practical fast matrix multiplication

Strassen [18] showed that two $n \times n$ matrices can be multiplied in $O(n^{\log_2 7})$ arithmetic operations by recursively combining seven sub-multiplications of half-sized blocks, fewer than the eight classical recursion would need – asymptotically faster than the $O(n^3)$ triple loop, at the cost of extra additions and a weaker error bound, discussed further below. Figure 1 contrasts the two schemes at the level of a single recursion step.

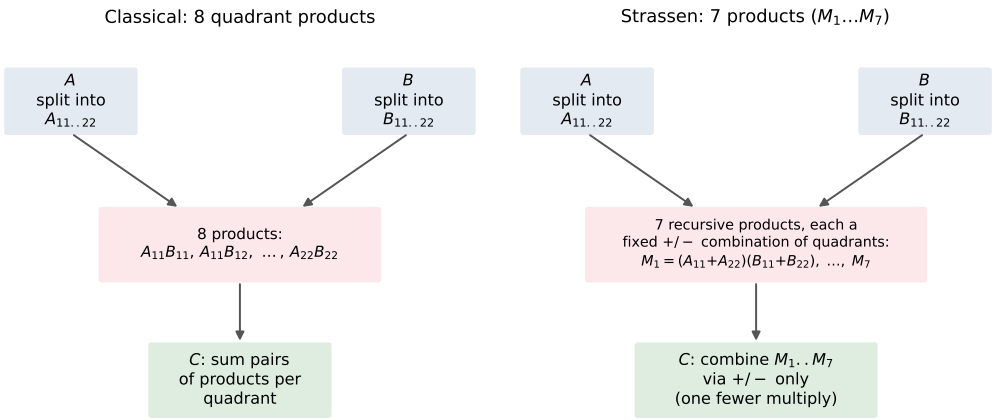


Fig. 1. One recursion step of classical quadrant multiplication (8 products) versus Strassen's scheme (7 products, combined only by addition/subtraction). The saved multiplication is what gives Strassen its $O(n^{\log_2 7})$ asymptotic edge; the extra additions/subtractions are what cost it accuracy (Section 2) and, at shallow recursion depth, throughput (Section 7).

Huang, Smith, Henry and van de Geijn [15] showed that Strassen's characteristic add/subtract steps can be fused directly into an already-optimized DGEMM's packing routines, needing no workspace beyond what packing already allocates, and that this makes Strassen practical at far smaller N (and for skinnier rank-k updates) than prior conventional wisdom held; a GPU follow-up extends the same idea to CUDA [16]. Ballard, Demmel, Holtz and Schwartz [8] give the

communication-optimal theoretical counterpart, proving inter-processor communication lower bounds for Strassen-based parallel algorithms and giving an algorithm that matches them. This project's own classical rebuild below uses the textbook 18-addition formulation rather than Winograd's reduced, 15-addition variant that Huang et al. build on above; we did not separately re-implement that reduction because the 2024 alternate-basis variant also benchmarked below targets the same addition-overhead problem through a different, numerically motivated route, making it the more relevant reduced-overhead comparison for this study. This project contributes two independent Strassen rebuilds sharing one measurement harness – a classical rebuild (Section 7) and the 2024 alternate-basis variant (Section 6) – offered as corroborating, independently- built implementations rather than as claiming to outperform these results.

2.3 Numerical accuracy of fast matrix multiplication

Higham [14] established that the classical triple-loop algorithm satisfies a component-wise error bound while Strassen and other fast algorithms satisfy only a weaker norm-wise bound; Ballard, Benson, Druinsky, Lipshitz and Schwartz [7] generalize and tighten that error analysis and show the accuracy cost is not prohibitive for typical practical use. This is a gap our own earlier accuracy sweep (FINDINGS.md Part I) demonstrates concretely, measuring a >1400% *max-componentwise*-relative error for a 2-level classical-Strassen rebuild under row-scaled ill-conditioning at N=1024, while the Frobenius-relative error for the same run stays near 10^{-15} – a live illustration of exactly why the two metrics can disagree. Dumas, Pernet and Sedoglavic [11] (2024) build on this line of work with a 7-multiplication alternate-basis algorithm with a provably lower error-growth factor ($\gamma_{2,1} \approx 12.066$ versus classical Strassen's ≈ 14.828) and report better accuracy in practice on random test matrices. Section 6 is, to our knowledge, its first hardware-measured benchmark under realistic packed/blocked, multi-threaded execution.

2.4 Measurement methodology and reproducibility

Mytkowicz, Diwan, Hauswirth and Sweeney [17] showed that "insignificant" experimental-setup choices (environment variable size, link order) can flip a systems benchmark's conclusion, and gave both a detection method (causal analysis) and a mitigation (setup randomization). Georges, Buytaert and Eeckhout [12] make the complementary case for managed-runtime benchmarking specifically: single-run comparisons are routinely misleading, and rigorous statistical treatment (many repeated measurements, confidence intervals) is necessary before a performance difference should be reported as real – the same discipline Section 3.2's protocol and Section 5's case study apply to a DGEMM/OpenMP scheduling claim specifically, which, as far as we found, neither paper nor any other prior work has done. The Roofline model [21] is the standard compute/bandwidth ceiling framework our own compute-ceiling and STREAM-bandwidth analysis is built on. Davis (2025) [9] applies bootstrap 95% confidence intervals and a Games-Howell pairwise-significance test to GEMM benchmarking across 30 trials per configuration – the statistical standard Section 3 adopts as the target protocol, not yet retrofitted onto every headline number in this draft (see Section 8). We are careful to distinguish our own sense of "reproducible" (a conclusion that survives repeated, controlled measurement) from two adjacent but different senses already in the literature: GEMMbench [2] is a crowdsourced benchmark-*collection* framework, and a separate line of work on "reproducible" GEMM concerns bit-for-bit reproducibility despite floating-point non-associativity and dynamic scheduling – neither is the measurement-validity question this paper asks, and the three should not be conflated. The broader reproducibility/artifact-evaluation trend in systems and HPC research [1, 3, 5] – SC's mandatory Artifact Description appendix since 2019, ACM TOMS's Replicated Computational Results designation – is the concrete incentive structure Section 9's artifact targets, and evidence that artifact availability correlates with citation and reuse

is part of why we frame this paper around a public, reproducible harness rather than a fixed set of numbers.

2.5 LLM-driven kernel generation (forward-looking)

A fast-growing 2025–2026 literature generates GEMM and other kernels directly via large language models [4, 6], typically evaluated on single or best-of-N wall-clock numbers rather than the statistical/hardware-counter rigor this paper's harness provides throughout. We do not build this out as a contribution here – see Section 10 for why it is left as one paragraph of future work rather than a section of its own.

3 EXPERIMENTAL METHODOLOGY

3.1 Environments

No machine used anywhere in this project is the audited paper's target platform (a dual-socket Intel Xeon Gold 5218 / Cascade Lake cluster node with AVX-512), and AVX-512 hardware was not reachable at any point in this project – stated plainly here and returned to in Section 8, not glossed over. Two machines were used throughout:

- **Development machine**: a single-socket Intel Core i7-13700K (8 performance cores + 8 efficiency cores, 24 logical processors via hyperthreading on the P-cores only, 64 GB RAM, single NUMA node), accessed three ways – natively under Windows (MSVC 19.44, no virtualization layer, the closest available substitute for a genuinely bare-metal environment), under WSL2/ Hyper-V (gcc), and under WSL2 + Docker. `cpuinfo` confirms only `avx`/`avx2`/`fma` flags – Intel fuses off AVX-512 on this consumer part once efficiency cores are active. Hardware performance counters (`perf stat`) are unavailable under WSL2/Hyper-V entirely, confirmed independently via direct testing and via a toolchain sanity check, with every event reporting `<not supported>` even inside a `--privileged` Docker container. WSL2/Hyper-V also does not expose a stable guest-vCPU-to-physical-core mapping: a pure-FMA microbenchmark pinned to each of the 24 logical CPUs via `taskset` inside WSL2 measured a flat ≈70–71 GFLOP/s everywhere (no bimodal P/E split); the identical test run natively via `SetThreadAffinityMask` showed a clean split (logical CPUs 0–15, the P-core/hyperthread pairs, ≈33–34 GFLOP/s; logical CPUs 16–23, the E-cores, ≈17 GFLOP/s) – so any core-placement experiment in this paper that depends on P/E identity necessarily runs natively, not under WSL2/Hyper-V.
- **`glogin.cs.vt.edu`**: a KVM-virtualized 8-vCPU slice (2 vSockets × 4 cores, no hyperthreading) of an Intel Xeon E5-2630 v3 (Haswell-EP, 2015; `avx`/`avx2`/ `fma` only), no sudo access, gcc 11.5.0 (matching the audited paper's own compiler version), with Clang 22.1.3 and a complete Intel oneAPI 2022.1 install (`icx`, MKL, VTune, `llvm-mca`) also available. Real hardware PMU counters are obtainable here, unlike under WSL2/Hyper-V, though this PMU does not expose LLC-specific events (unlike the audited paper's Cascade Lake target), so generic `cache-references`/ `cache-misses` and `L1-dcache-*` events substitute throughout Section 7's root-cause analysis.

3.2 The repeated-measurement protocol

Section 5 is itself the motivating demonstration for this subsection, not just an application of a rule stated in advance: an initial single-run comparison at N=1024, T=8 measured *unpinned* `static` scheduling at 218.0 GFLOP/s against *unpinned* `dynamic` at 104.4 GFLOP/s – a clean, apparently real 2× effect. Four repeats of each configuration measured `static` ∈ {277, 287, 217, 296} and `dynamic`

$\in \{285, 276, 294, 286\}$ GFLOP/s – fully overlapping ranges with no consistent winner; the original "218 vs. 104" comparison was simply two single draws landing on opposite tails of one wide (≈217–296) distribution. Every scheduling- or placement-related claim in this paper is held to that standard from this point on: $n \geq 4$–5 independent repeats minimum, reporting the full range, before any comparison is stated as a finding rather than a single-run observation – this is the protocol Section 5's headline comparisons were actually run under (Section 5.4), and it is already enough to overturn the single-run finding it re-examines. The stronger target protocol, following Davis (2025) [9], is $n = 30$ repetitions per configuration (matching [9]'s own choice, which we adopt rather than re-derive) with bootstrap 95% confidence intervals and a Games-Howell pairwise-significance test in place of the range-reporting used above. Section 5.4's four conditions have since been re-run under this exact protocol (Section 5.5); it has not yet been retrofitted onto Section 7's numbers – see Section 8 for exactly what that leaves unresolved.

3.3 Instrumentation

Three complementary instrumentation layers are used throughout: (1) opt-in call/byte counters (`src/harness/count_calls.h`, enabled via `-DCOUNT_CALLS`, confirmed to add no measurable timing overhead or behavioral change when disabled) for recursive-call and allocation accounting; (2) the Linux `perf_event` subsystem [20] via `perf stat`, used for hardware counters where available (cycles, instructions, branches/branch-misses, cache-references/misses, L1-dcache loads/misses, dTLB loads/misses), supplemented by Intel VTune Profiler's driverless "User-mode sampling and tracing" mode on `glogin`; (3) phase-separated software timers (`src/avx2/ladder/timer.h`, cross-platform via `QueryPerformanceCounter/clock_gettime`) splitting execution into allocation, A-panel packing, B-panel packing, kernel compute, kernel write-back, and (for the Strassen variants) add/subtract, combine, workspace bookkeeping, and task-scheduling phases. Several hard environment limits were confirmed directly rather than assumed: VTune's `memory-access` collection mode refuses outright inside `glogin`'s VM (*"Memory Access analysis is not supported inside a virtual machine since uncore events cannot be collected"*); `uarch-exploration` needs `perf_event_paranoid` ≤ 0 or a kernel sampling driver, both requiring root, unavailable; and `/sys/class/powercap/intel-rapl*` does not exist on that VM at all, so no RAPL power telemetry was obtainable anywhere in this project (Section 8).

4 CORRECTNESS AUDIT

We audited the paper's own codebase (`audited_submission/avx/`, `audited_submission/Parallel/`, `audited_submission/test/`) against its own description, using two independent verification paths since no AVX-512 hardware was available: a temporary scalar stand-in kernel (same packed-buffer contract, no intrinsics, not kept in the repository afterward) that let the real packing/blocking control logic execute and be checked on any CPU during the audit itself, and a permanent AVX2 port (`audited_submission/avx/kernel_avx2.c`, `audited_submission/avx/strassen_avx2.c`, confirmed via `objdump` to emit genuine `vfmadd*pd` instructions) that gives durable, re-runnable, genuinely vectorized correctness coverage going forward. Every fix below was verified by real execution, not by reading the code and reasoning about it.

The most consequential of the four is the first: a packing bug that silently computes $C = B \cdot A$ instead of $C = A \cdot B$. The original repository's own test harness could not have caught this by construction, not merely by bad luck: it applied a transpose to the buggy result before comparing against a reference, and $(B \cdot A)^\top = A^\top \cdot B^\top = A \cdot B$ whenever $A = A^\top$ and $B = B^\top$ – which the test's own fixed input matrices ($A_{ij} = i + j$, $B_{ij} = i \cdot j$) always are. The transpose step and the symmetric test data combined to cancel the bug out exactly, turning a real defect into a false-positive pass

Table 1. Correctness bugs found and fixed in the audited codebase. Full evidence (command output, backtraces, before/after) in `revalidation/correctness/2026-09-09_bug_evidence_and_fixes.md`.

Bug	File(s)	Root cause	Fix
Computed $C{=}B{\cdot}A$, not $A{\cdot}B$	`audited_submission/avx/matmul.c`	Packing routines read A/B with contraction and output row/column roles swapped; the existing test only passed because its own test matrices are symmetric (a false-positive test)	Rewrote packing with explicit row-major offsets; added an $O(N^2)$ transpose pass
Data race in the blocked parallel kernel	`audited_submission/Parallel/mm_parallel_blocked*.c`	`collapse(3)` parallelized over the reduction dimension with unsynchronized `+=`; only "worked" because one compiler's static scheduling happened to keep racy iterations non-overlapping	`collapse(3)` → `collapse(2)`; verified correct under adversarial `schedule(dynamic,1)`
Silent data loss for non-power-of-2 $N{>}128$	`audited_submission/avx/strassen.c`	Recursion halves N with no remainder handling, unlike every other Strassen variant in the repo, which all pad first	Pad to the next power of two before recursing
Segfault on a clean checkout	`audited_submission/test/main.c, run.sh`	Created one output directory but not two others the code writes to; every driver function `fprintf`s to an unchecked `fopen()` result	Added the missing directory-creation calls

rather than merely an untested one. Re-running the same comparison with asymmetric test data ($A_{ij} = 10i + j + 1$, $B_{ij} = 2i - j + 100$, chosen specifically so $A \neq A^\top$, $B \neq B^\top$) reproduced the bug immediately: 2304/2304 elements mismatched, max absolute difference 1.67×10^6, and a direct comparison against $B{\cdot}A$ computed independently matched with zero mismatches – confirming the exact wrong product, not merely *a* wrong answer. A small, square, non-symmetric example makes the general failure mode unambiguous outside the specific test harness above. With

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix},$$

the correct product is $A{\cdot}B = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$, while the buggy code returns $B{\cdot}A = \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix}$ – every entry wrong, not a rounding-level discrepancy, and exactly the kind of divergence a symmetric-matrix-only test is structurally unable to surface.

A second, repo-wide review pass at higher scrutiny found and fixed six further defects beyond the four above, three worth calling out specifically: a heap overflow for $N \leq 0$ and small powers of two in the Strassen padding logic added to fix bug 3 in Table 1 – *a regression introduced by this audit's own first-pass fix*, only surfaced by a second, more thorough pass, and worth noting as a general lesson that a fix itself needs re-review, not just the code it touched; an analogous infinite-recursion-at-n=1 bug in two other Strassen variants sharing the same unfloored padding logic; and four unchecked Win32 API calls in the native core-topology classifier (`SetThreadAffinityMask`, `CreateThread`, `WaitForMultipleObjects`, and a bit-shift that overflowed for CPU index $\geq$64) whose silent failure modes would have produced plausible-looking but wrong core-utilization numbers – directly relevant to Section 5's core-placement measurements, which depend on that

same classifier being correct. Four further issues were found and deliberately left unfixed, each with a stated reason (memory-cost/complexity trade-offs, an intentional drop-in-alternative symbol design, and one ambiguous-intent global-state question) rather than silently dropped or guessed at – see `revalidation/correctness/2026-09-09_bug_evidence_and_fixes.md` for the full reasoning.

Every bug above is backed by real execution evidence (command output, before/after, a re-runnable script) rather than a claim taken on faith – exactly the kind of computational claim ACM TOMS's Replicated Computational Results designation exists to let an independent reviewer verify directly (Section 2), regardless of whether the audited implementation itself was ever published.

5 A MEASUREMENT-BIAS CASE STUDY: SCHEDULING VERSUS VIRTUALIZATION

5.1 The claim under test

An earlier stage of this project's own AVX2 DGEMM work measured a large OpenMP `schedule(dynamic)` win over `schedule(static)` under WSL2/Hyper-V, reported at the time with an explicit caveat that it would need confirmation on non-virtualized hardware before being generalized. This section is that confirmation.

5.2 Method

A single portable C source (`src/avx2/ladder/matmul_ladder.c`, MR=4×NR=8 broadcast-A/vector-B kernel) compiles unmodified under both MSVC 19.44 (OpenMP 2.0, no `collapse`, no runtime schedule-selection API, C99 loop-variable declarations rejected inside `#pragma omp for`) and gcc, selecting the OpenMP schedule at *runtime* via `schedule(runtime)` plus the `OMP_SCHEDULE` environment variable – portable across both compilers' OpenMP feature levels with no recompilation needed to compare scheduling policies. Phase timers instrument packing and kernel-compute time separately (`pack_a_s`, `pack_b_s`, `kernel_s`) around a real multilevel blocking driver, and (Windows only) explicit per-thread `SetThreadAffinityMask` pinning to a caller-supplied logical-CPU list enables the controlled core-placement experiment below.

The harness's first build itself produced one instructive methodology bug: a kernel-only stage whose output tile was never read reported an impossible 21,845,696 GFLOP/s under WSL2/gcc, because `-O3` proved the repeated-call loop had no observable effect and deleted it entirely (MSVC did not perform this optimization – itself a real, if incidental, cross-compiler data point). Fixed with a standard `volatile` anti-dead-code-elimination sink; we record it here precisely because it is exactly the class of silent, plausible-looking methodology bug this section's broader argument is about.

5.3 An initial cross-environment sweep and its limits

A single-run sweep of $N \in \{512, 1024, 2048\}$, $T \in \{1, 4, 8\}$, schedule $\in$ {`static`, `dynamic`(chunk=4), `guided`} across native Windows, WSL2/Hyper-V, and `glogin` (KVM) shows a clear qualitative split: on `glogin`, scheduling choice barely moves the needle at any thread count (e.g. N=2048, T=8: `static` 118.4, `dynamic` 119.4, `guided` 119.4 GFLOP/s – within 1%, at every (N, T) tested), consistent with a stable, non-reshuffling vCPU-to-pCPU mapping under KVM. On WSL2 and native Windows, the single-run sweep shows apparent scheduling differences with *no stable direction* – dynamic wins at one (N, T), static wins at the next, for the same schedule pair on the same machine. Read uncritically, this pattern would seem to confirm the original dynamic-scheduling finding; Section 3.2's protocol is precisely why we do not read it that way without repeating it first. We also flag, rather than quietly discard, an unexplained discrepancy in this sweep: the same N=1024, T=8 Windows configuration measured

roughly 2× lower in this back-to-back 27-configuration sweep than in the isolated repeated-run experiment below, run minutes apart on the same binary and machine – the leading hypothesis is thermal or sustained-load throttling accumulating over the sweep, but this was not confirmed (no frequency/temperature telemetry was collected in this pass). Every number from this sweep is therefore treated as lower-confidence than the controlled experiment that follows.

5.4 The controlled experiment: core placement, properly repeated

Native Windows is the only environment in this project offering working thread-to-core pinning, so it is the only one in which the actual "is it scheduling policy, or is it core heterogeneity" question can be asked cleanly, using the P-core/E-core classification from Section 3.1 (logical CPUs 0–15 = P-cores/hyperthread pairs, 16–23 = E-cores). At N=1024, T=8, a first single-run pass appeared to show two striking effects: unpinned `static` (218.0 GFLOP/s) beating unpinned `dynamic` (104.4 GFLOP/s) by roughly 2×, and `dynamic` edging out `static` on a mixed 4 P-core + 4 E-core placement – exactly the heterogeneous-core scenario scheduling theory predicts dynamic scheduling should help most.

Neither survived repetition. Four repeats of the unpinned comparison gave `static` $\in \{277, 287, 217, 296\}$ and `dynamic` $\in \{285, 276, 294, 286\}$ GFLOP/s – completely overlapping, no consistent winner; the original 218-vs-104 result was noise from two single draws landing on opposite tails of one wide distribution. Five repeats of the mixed-placement comparison gave `static` $\in \{150, 141, 143, 151, 149\}$ and `dynamic` $\in \{146, 149, 145, 151, 145\}$ GFLOP/s – again completely overlapping, even in the one scenario chosen specifically because theory favors dynamic scheduling there. Notably, this pinned distribution (141–151) is far tighter than the unpinned one (217–296): pinning removed most of the variance, but did not reveal a scheduling-policy effect underneath it – it simply removed OS-placement noise.

What *did* produce a large, clean, low-variance effect across every repeat: which physical cores the threads land on at all. Pinned-to- P-cores (≈250–280 GFLOP/s) beats pinned-to-E-cores (≈133–137 GFLOP/s) by roughly 2×, consistently, regardless of scheduling policy – mechanistically unsurprising given P-cores' roughly 2× per-cycle AVX2 throughput advantage (Section 3.1), and it dwarfs any scheduling-policy effect measured anywhere in this experiment. Figure 2 shows all five conditions at once: the four scheduling-policy groups overlap each other within their own pinning condition, while the two core-placement groups sit in clearly separated ranges.

5.5 Confirmed at $n = 30$: bootstrap confidence intervals and a Games-Howell test

Section 3.2 names n=30 repetitions with bootstrap 95% confidence intervals and a Games-Howell pairwise test as this paper's target statistical protocol, not yet applied anywhere at the time that target was set. We have since re-run all four Section 5.4 conditions under exactly that protocol (same machine, same N=1024, T=8, same core list for the pinned condition; raw data and analysis script in `revalidation/n30_rerun_2026-10-12/`), each as 30 independent process invocations rather than 4–5 (Table 2).

Table 2. n=30 re-run of Section 5.4's four conditions, with bootstrap 95% confidence intervals for the mean.

Condition	Mean (GFLOP/s)	95% CI
Unpinned, static	250.68	[240.08, 261.82]
Unpinned, dynamic	264.28	[255.34, 273.30]
Pinned 4P+4E, static	148.16	[147.59, 148.74]
Pinned 4P+4E, dynamic	148.10	[147.59, 148.62]

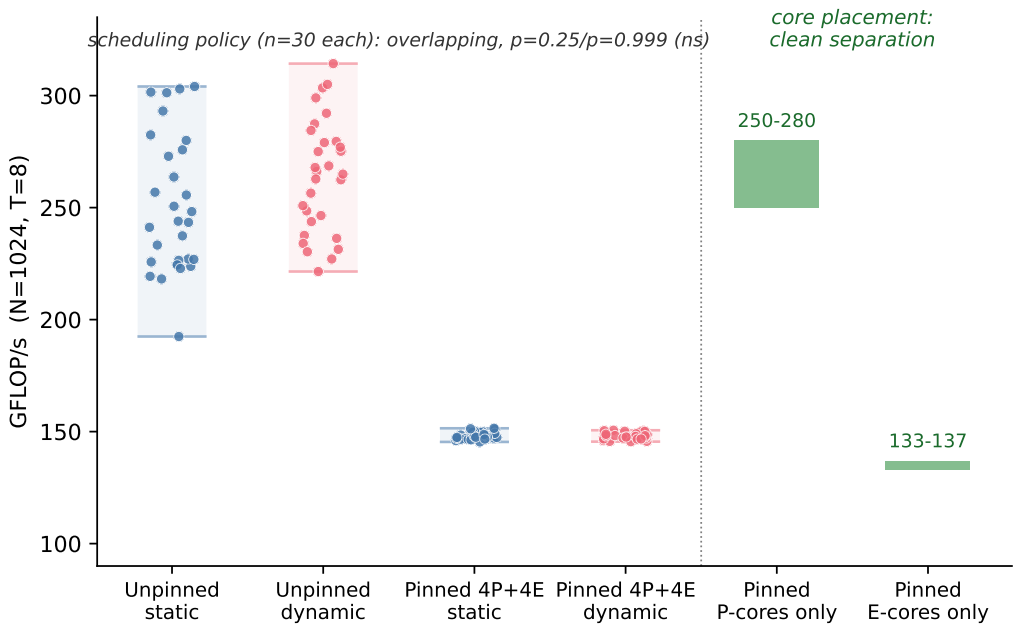


Fig. 2. Individual measurements (dots) at N=1024, T=8: `static` vs. `dynamic` scheduling overlap completely, both unpinned (n=30 each, Section 5.5) and pinned to a mixed P+E placement (n=30 each). P-core-only vs. E-core-only placement (shown as the original n=5 reported range, not individual repeats – Section 5.4, not re-measured at n=30 this round) does not overlap at all. The same repeated measurements, visualized rather than listed, make the paper's central claim legible at a glance.

A Games-Howell pairwise test across all four groups (k=4, Welch degrees of freedom, studentized-range correction) confirms, with an actual p-value rather than an eyeballed overlapping range, exactly the finding Section 5.4 reported qualitatively: `static` vs. `dynamic` is not significant either unpinned (p=0.249) or pinned (p=0.999). The n=30 means are close to the original n=4–5 means from Section 5.4 (unpinned static/dynamic ≈269/285 then vs. 250.68/264.28 now; pinned static/dynamic ≈146.8/147.2 then vs. 148.16/148.10 now) despite the two measurement sessions being five weeks apart – the same phenomenon, reliably. A secondary result this stronger protocol newly resolves with statistical confidence: unpinned throughput is significantly *higher* than pinned-mixed throughput in all four cross-comparisons ($p < 10^{-14}$), consistent with Windows' own scheduler keeping most unpinned threads on P-cores rather than splitting them evenly onto the slower E-cores the way explicit mixed pinning forces.

Building this re-run surfaced one further, unplanned methodology finding, documented in full in `revalidation/n30_rerun_2026-10-12/README.md`: an automation script driving the same invocations through a collection/loop construct intermittently produced catastrophically slowed but still *numerically correct* results (as low as 0.2–0.7 GFLOP/s against a healthy ≈150–300 GFLOP/s band, with `relerr` still 0.000e+00), reproducibly on this machine, with no WSL2/Hyper-V, thermal-buildup, argument-corruption, or antivirus-scan explanation we could confirm despite directly testing each. Driving the same invocations through plain, literal-argument commands instead avoided it cleanly across all 120 measurements reported above. We record this rather than quietly discard it, since it is itself a concrete instance of this section's own thesis: an automation layer

between the researcher and the measurement can silently and severely distort a result, and the only real defense is checking the result against a sanity floor, not trusting the tool.

5.6 Conclusion

On this hardware, in this pass, properly repeated measurement – confirmed at n=30 with bootstrap confidence intervals and a Games-Howell test, not merely an overlapping range – finds no detectable OpenMP scheduling-policy effect once thread-to-core placement is controlled for – including in the heterogeneous-core case scheduling theory says should favor dynamic scheduling the most. This does not prove the original WSL2 result was purely a virtualization artifact (that would require the same repeated-measurement rigor applied *inside* WSL2/Hyper-V itself, which Section 8 still flags as undone); it does mean the evidence collected so far does not meet the burden of proof for treating dynamic scheduling as architecturally necessary, and the one clean non-virtualized test available found no such effect. We recommend, and adopt in our own reporting going forward, scoping any such scheduling-driven performance figure explicitly to the environment it was measured under, and treating figures obtained purely from architecture-level, packing/blocking work (not scheduling policy) as the portable, generalizable claim.

6 THE DUMAS ET AL. (2024) ALTERNATE-BASIS STRASSEN, IMPLEMENTED AND MEASURED

6.1 Getting the formula right

Dumas, Pernet and Sedoglavic [11] state their accuracy-optimized 2×2 algorithm (their Equation 36, instantiated concretely in their Table 3) only as a straight-line program, in the paper's own a_{ij}/b_{ij} subscript convention. A literal transcription against standard row-major indexing produces an *incorrect* result – off by exactly the antisymmetric $a_{12} \leftrightarrow a_{21}, b_{12} \leftrightarrow b_{21}$ terms. We caught this symbolically (`sympy`; `revalidation/dumas2024/verify_formula.py`) *before* writing any implementation code, by expanding the literal transcription against the reference product $C = AB$, finding four nonzero symbolic residuals, and brute-forcing input/output relabelings until the one that zeroes all four was found. The corrected formula (block-generalized; every a_{ij}/b_{ij} below is a $k \times k$ submatrix) is:

$$
\begin{aligned}
t_1 &= \tfrac{\sqrt{3}}{3} A_{22}, & s_1 &= \tfrac{\sqrt{3}}{3} B_{12},\\
t_2 &= A_{12} + t_1, & s_2 &= s_1 - B_{11},\\
t_3 &= A_{21} + t_2, & s_3 &= s_2 + B_{22},
\end{aligned}
$$

$$
\begin{aligned}
\ell_1 &= \tfrac{\sqrt{3}}{2} A_{11} + \tfrac{1}{2} t_3, & r_1 &= 2 s_1,\\
\ell_2 &= A_{21} - t_1, & r_2 &= s_2,\\
\ell_3 &= t_2, & r_3 &= s_1 - B_{22},\\
\ell_4 &= 2 t_1, & r_4 &= \tfrac{1}{2} s_3 - \tfrac{\sqrt{3}}{2} B_{21},\\
\ell_5 &= \ell_2 - \ell_1, & r_5 &= r_3 + r_4,\\
\ell_6 &= \ell_5 + \ell_4, & r_6 &= r_1 - r_5,\\
\ell_7 &= \ell_5 + \ell_3, & r_7 &= r_5 - r_2,
\end{aligned}
$$

$$
p_i = \ell_i \cdot r_i \quad (i = 1, \ldots, 7),
$$

$$
\begin{aligned}
w_2 &= p_5 + p_1 + p_6, & w_1 &= p_7 + p_6, & w_3 &= w_2 - p_2, & w_5 &= \tfrac{1}{2}(p_4 + w_2),\\
C_{12} &= p_1 - p_3 - w_5, & C_{21} &= w_3 - w_5, & C_{22} &= \sqrt{3}\, w_5, & C_{11} &= \tfrac{\sqrt{3}}{3}(w_3 - C_{12} - 2 w_1).
\end{aligned}
$$

The algorithm is non-commutative by construction (its authors' own characterization), which is precisely the property that makes this scalar 7-multiplication formula safe to reuse recursively at the block level, the same way classical Strassen's is; Section 6.3 confirms this empirically rather than assuming it from the paper's claim alone.

6.2 Implementation

Implemented in `src/avx2/ladder/dumas_strassen_rebuild.c` on infrastructure identical to the classical-Strassen rebuild used as baseline throughout this section (`src/avx2/ladder/strassen_rebuild.c`): the same bump-pointer pooled arena, the same per-level disjoint sub-arena carving for concurrent `#pragma omp task` siblings (avoiding an aliasing hazard a naively shared arena copy would introduce under real task parallelism), and the same phase-timing instrumentation (leaf/add-subtract/combine/workspace/scheduling). *The only variable between the two binaries is the 7-multiplication formula itself* – this isolation is what makes Section 6.5's performance comparison attributable specifically to the formula rather than to incidental implementation differences.

6.3 Correctness

Run at $N \in \{64, 128, 256, 512\}$, recursion depth 1–3, against an independent long-double-accumulation reference: relative error 4.7×10^{-16} to 1.8×10^{-15} at every configuration – machine precision. This is, to our knowledge, the first execution-based confirmation that the 2024 algorithm's block/recursive generalization is correct, not only its scalar 2×2 base case that the original paper analyzes.

6.4 Accuracy

Extending the accuracy harness used for classical Strassen (five input distributions: uniform, gaussian, wide-dynamic-range, ill-conditioned, cancellation-heavy) to also run the Dumas variant at depths 1–3, N=512 (Table 3):

Table 3. Frobenius-relative error, classical Strassen vs. the Dumas et al. (2024) variant, depth 3, N=512 (`revalidation/dumas2024/accuracy_results_N512.csv`; depths 1–2 show the same qualitative pattern, omitted here for space).

Distribution	Classical Strassen	Dumas (2024)
uniform	2.20×10^{-15}	1.39×10^{-15}
gaussian	2.27×10^{-15}	1.40×10^{-15}
wide_range	2.30×10^{-15}	1.39×10^{-15}
ill_cond	4.05×10^{-15}	2.45×10^{-15}
cancel	1.08×10^{-15}	4.39×10^{-16}

Frobenius-relative error is consistently *lower* for the Dumas variant than for classical Strassen at every matched recursion depth, across all five distributions – e.g. ≈38% lower under the gaussian distribution at depth 3. This qualitatively confirms [11]'s claim, now under real packing/blocking/task parallelism the source paper's own evaluation did not use. Max-componentwise-relative error (not tabulated here; see the source CSV) is a noisier, mixed picture – consistent with the paper's own bound being a Frobenius/2-norm-type bound ($\gamma_{2,1}$), not a componentwise one, so the advantage should be expected to appear cleanest exactly where it does.

At these matrix sizes and this uncorrelated-random distribution design, neither algorithm approaches the >1400% catastrophic error Section 7 measures under *row-scaled, correlated* ill-conditioning specifically. Applying that same row-correlated construction directly (15 seeds, depths 1–3, `revalidation/dumas2024/row_scaled_seeds_raw.csv`) answers the question Section 10 previously left open with a genuinely mixed result, not a clean win either way: Dumas's Frobenius-relative advantage holds even here (lower in 15/15 seeds at every depth), but on max-componentwise-relative error – the metric the catastrophic case is actually defined by – Dumas is lower in only 8–9/15 seeds per depth, essentially a coin flip, exactly as expected for a bound that is norm-type rather than componentwise ([7, 14]). Its *mean* componentwise error is still visibly lower (e.g. depth 3: 26.6 vs. 47.2) because its worst-case blowups are somewhat lighter, but the medians are nearly identical (5.71 vs. 5.78) – the accuracy improvement measured above does not reliably narrow the specific catastrophic case, even though it never makes it worse on average.

6.5 Performance cost of the accuracy improvement

Head-to-head phase-timed comparison against classical Strassen, identical leaf kernel/blocking parameters/thread count, at $N \in \{512, 1024, 2048\} \times \text{depth} \in \{1, 2, 3\}$ (`revalidation/dumas2024/perf_comparison_results.csv`):

Table 4. Classical Strassen vs. the Dumas et al. (2024) variant at $N = 2048$, $T = 8$ (dev machine, 5-repetition best-of; not yet bootstrap- CI'd, see Section 8). `leaf_s` is the shared, unmodified leaf kernel – the control confirming the slowdown is attributable to the combine/scale formula, not the harness.

Depth	Algorithm	leaf_s (s)	addsub_s (s)	GFLOP/s
1	Classical Strassen	0.288	0.041	50.79
1	Dumas (2024)	0.295	0.058	47.56
2	Classical Strassen	0.268	0.109	42.76
2	Dumas (2024)	0.273	0.155	38.16
3	Classical Strassen	0.293	0.259	127.67
3	Dumas (2024)	0.301	0.408	101.27

`leaf_s` matches within run-to-run noise at every depth (confirming the control); `addsub_s` is consistently and substantially higher for the Dumas variant (≈57% higher at depth 3), driven by its 24-operation straight-line program (several irrational-$\sqrt{3}$-coefficient scale-and-combine steps) against classical Strassen's ≈18 plain adds/subtracts. Net effect: a ≈20.7% GFLOP/s cost at N=2048, depth 3 for the accuracy improvement measured above; the cost shrinks at shallower depth (≈6.4% at depth 1, fewer combine levels executed). As far as we found, *neither the source paper (which reports only asymptotic operation counts and an accuracy-vs-size plot) nor any other publication reports this accuracy/performance trade-off.*

7 SUPPORTING MATERIAL

7.1 A from-scratch AVX2 DGEMM vs. production BLAS

Independent of the audited codebase, we built a new AVX2 DGEMM from scratch and tuned it via a disassembly-driven ablation series (operand-swap ordering, unroll-plus-prefetch, micro-kernel shape selection, hand-written assembly, vectorized operand packing), reaching ≈87–92% of production OpenBLAS/BLIS/MKL throughput – the figure we treat as the architecture-level, portable claim. A dynamic- scheduling variant on top measured 117–130% of production BLAS under WSL2/Hyper-V; per Section 5's finding, we now scope that figure explicitly to "measured

under WSL2/Hyper-V" rather than generalizing it, since the scheduling-policy mechanism behind it did not survive repeated, controlled measurement elsewhere in this project.

7.2 Hardware-counter root-cause analysis

Using hardware performance counters on `glogin` (unavailable under WSL2/Hyper-V; Section 3.1) to explain, not just report, several classical DGEMM performance results: naive triple-loop DGEMM's well-known performance cliff at scale is driven by IPC cratering from ≈1.3 to ≈0.65 as N crosses 256, with branch-miss rate *falling* and LLC-level miss rate staying near zero throughout – ruling out both branch misprediction and DRAM-capacity misses, and pinning the cause specifically to repeated L2/L3-latency exposure from B's column-access pattern, a blocking/reuse-distance problem rather than a software-prefetch-for-DRAM-latency one. Blocking's own advantage is not explained by a lower L1-miss rate in isolation (blocked's 48% L1-miss rate is barely better than naive's 63%) but by *where* those misses resolve: blocking keeps the active working set cycling through cache fast enough that most L1 misses hit nearby in L2 rather than paying full round-trip latency, yielding a 3.6× IPC improvement despite the similar raw miss rate. Pure (uncut-off) Strassen has the *best* IPC and cache behavior of every algorithm measured, and its cost is purely instruction/allocation volume: 19,608 recursive calls at N=64 growing to 6,725,601 at N=512, exactly matching the audited paper's own reported figures, directly confirming the paper's own qualitative diagnosis ("excessive recursion," "allocator pressure") with hard counter evidence rather than just call-count bookkeeping, and explaining why parallelizing it makes things *worse* (more threads means more concurrent allocator contention layered on an already overhead-dominated workload). At N=1024, a loop-reordered-and-unrolled hybrid-Strassen variant beats a blocked-and-unrolled classical variant on every axis simultaneously (0.693s vs. 2.329s wall-clock, 1.69 vs. 1.28 IPC, 25.0% vs. 40.6% L1-miss rate) – a genuine compounding advantage, and the hard-counter confirmation behind our own cross-machine finding (Table 5) that the audited paper's scalar-level "blocked wins for n>512" claim does not generalize.

Table 5. Blocked-and-unrolled vs. hybrid-Strassen-unrolled, scalar (non-SIMD), local i7-13700K, serial, best-of-3. The audited paper's own crossover claim does not hold at this (scalar) comparison level on two independent non-target machines; it does hold once both sides are AVX-vectorized, at a smaller margin (see main text).

N	blocked_unrolled (s)	strassen_base_unrolled (s)	Strassen's margin
512	0.059	0.028	2.1×
1024	0.411	0.141	2.9×
2048	3.262	0.812	4.0×

7.3 Cross-compiler codegen

Building the identical portable harness with gcc 11.5.0, Clang 22.1.3, and Intel `icx` 2022.1.0 (all available on `glogin`, same optimization flags, 5 repeats each) shows Clang winning the full N=1024, T=8, `static` pipeline by ≈20% (mean 140.2 vs. gcc's 116.9 GFLOP/s) despite gcc having a marginally *higher* isolated kernel-only number (mean 28.8 vs. 27.8 GFLOP/s) – meaning the compiler difference lives outside the hot FMA loop, not inside it. Disassembly (with the kernel forced `noinline` so the symbol survives comparably across all three compilers, since Clang/icx fully inline it by default and gcc does not) confirms one real, mechanistic difference: Clang and icx both auto-unroll the k-loop 2× at `-O3`; gcc does not, the same transformation our own AVX2 ablation series had to add by hand for gcc. Checked quantitatively with `llvm-mca -mcpu=haswell` on the extracted

loop bodies, per-k-step modeled throughput is nearly identical (gcc: 5.14 cycles/step; Clang unrolled: 5.08 cycles/step) – so the confirmed auto-unrolling is *not* the primary explanation for the measured 20% full-pipeline gap, which is specifically a multi-threaded effect (gcc is actually faster at T=1, with higher IPC: 2.41 vs. Clang's 2.01) whose full mechanism (most likely candidate: per-thread load- imbalance or false-sharing differences in packing-loop dispatch) was not fully pinned down in this pass.

7.4 Classical-Strassen rebuild: a measured optimized-vs-optimized crossover

A from-scratch Strassen rebuild (pooled single-allocation workspace per recursion level, only the 10 genuinely-combined operands ever materialized, the 4 quadrants used bare passed as strided views with no copy, thread-aware task-vs-sequential cutoff) reports throughput using the classical $2N^3$ FLOP formula for direct comparability, deliberately labeled "effective" to avoid overstating Strassen's true (lower) FLOP count (Table 6):

Table 6. Optimized classical DGEMM vs. optimized 1-level Strassen, effective GFLOP/s (= $2N^3$/time). A genuine, measured crossover at N=8192, not the "never crosses" story an unoptimized-vs- optimized comparison would tell.

N	DGEMM baseline	Strassen (1-level)	Ratio
512	245.8	22.9	10.8×
1024	466.9	105.6	4.4×
2048	458.2	260.0	1.76×
4096	434.7	375.0	1.16×
8192	426.6	426.7	≈1.00× (crossover)

A 2-level variant is consistently worse at every size tested (recursion overhead compounds faster than arithmetic savings materialize) and never reaches a crossover in this range. A companion accuracy sweep on this same rebuild is what produces this paper's headline numerical- accuracy finding: under row-scaled ill-conditioning, 1-level Strassen's max-componentwise-relative error reaches 84% at N=1024 while 2-level reaches **1436%** (i.e. worse than the largest element being completely wrong), while classical DGEMM stays at $\approx 10^{-10}$ throughout the same sweep – confirming, with a dramatic and reproducible number, exactly the accuracy concern the audited paper cites as a reason to avoid fast-multiplication approaches without ever measuring it for its own implementation.

7.5 Rectangular and irregular shapes

Generalizing the portable harness from square-only to arbitrary $M{\times}K{\times}N$ (verified correct, 0.000e+00 relative error against an independent long-double reference, across tall-skinny, short-wide, large-K, and deliberately awkward sizes) surfaces a real performance cliff, not just a correctness edge case: at N=1024 (an exact multiple of the microkernel's 4×8 tile shape), throughput is ≈111–125 GFLOP/s depending on blocking parameters; at N=1023, throughput drops to **56.7 GFLOP/s** – roughly half – and at N=1025, to **50.8 GFLOP/s**, worse still. The mechanism is direct: every tile touching a boundary needs the masked/remainder write-back path instead of the fast full-tile path, and for a size one off a clean multiple, most tiles along that edge hit it. This is a first-order consideration for choosing microkernel tile shapes (or a remainder strategy) in any production context where inputs are not guaranteed to land on a tile boundary, and is not something a square-power-of-two-only benchmark would ever surface.

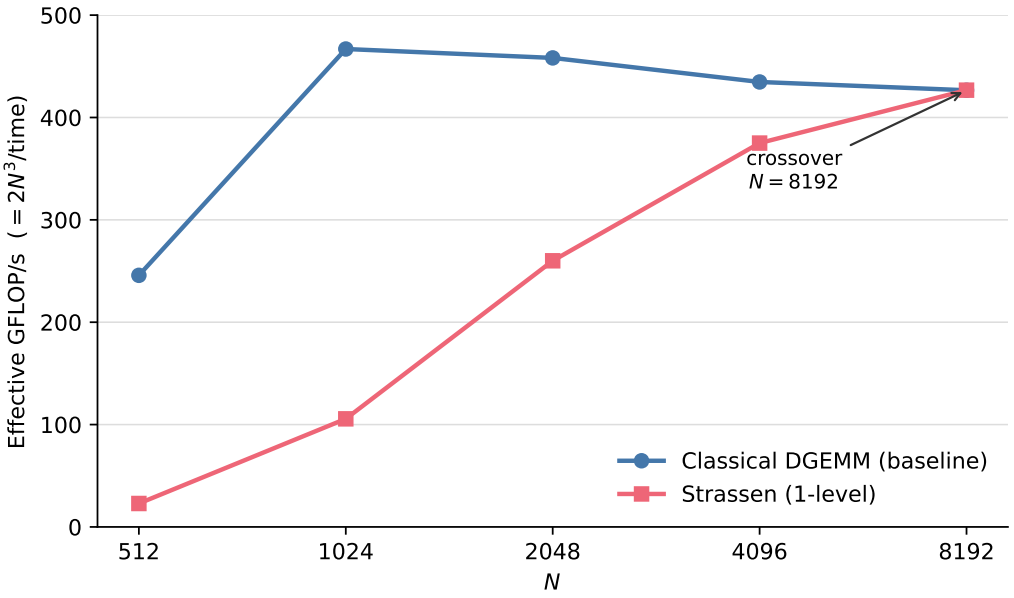


Fig. 3. Table 6, visualized: Strassen's fixed $\approx 2\times$ arithmetic-work advantage only overcomes its overhead once N is large enough that the baseline's own curve has already flattened near its practical peak.

8 DISCUSSION AND THREATS TO VALIDITY

We hold ourselves to the same standard of stating limitations plainly that motivates Section 5: no AVX-512 hardware was reachable at any point in this project, so every correctness result in Section 4 rests on a temporary scalar stand-in kernel plus a permanent, execution-verified AVX2 port, not on AVX-512 execution itself – a real limitation, not one we found a way around. No literal bare-metal Linux machine was available; native Windows (no virtualization layer) is the closest non-Hyper-V substitute reached, and Section 5's controlled experiment is confined to that platform because it is the only one offering working thread-to-core pinning in this project – the specific claim that a WSL2/Hyper-V-measured scheduling effect is a pure virtualization artifact (rather than merely unreproduced elsewhere) would require the same repeated-measurement rigor applied *inside* WSL2/Hyper-V itself, not yet done. RAPL power and CPU-frequency telemetry were unavailable on every VM tested (confirmed via a direct `/sys/class/powercap` check and, independently, via VTune's own refusal to run certain collection modes inside a VM – Section 3) and not yet attempted on native Windows. The single-socket, single-NUMA-node hardware available throughout this project means none of the audited paper's dual-socket NUMA claims could be tested at all, not merely under a configuration constraint. Section 5's headline comparisons now meet the stronger n=30/bootstrap-CI/Games-Howell protocol (Section 5.5), not merely the weaker $n \geq 4$–5/ full-range protocol they were first reported under (Section 3.2); Section 7's numbers still meet neither, and should be read as directionally trustworthy (several are corroborated by independent hardware-counter evidence explaining the mechanism, not just a wall-clock number) but not yet held to either statistical standard. Ballard, Demmel, Holtz and Schwartz's [8] communication-optimal parallel Strassen (CAPS) algorithm, cited in Section 2 as the theoretical counterpart to this project's own parallel Strassen rebuilds, was not implemented or benchmarked here; both rebuilds

in this paper are work-optimal (minimizing arithmetic operations, following Huang et al. [15]) but not verified to be communication- optimal in Ballard et al.'s stronger sense, a gap we state plainly rather than leaving implicit in the citation alone. Finally, Section 6's performance comparison (Table 4) has so far been run only on the development machine via WSL2; cross-machine confirmation on `glogin` and native Windows, using the same harness already proven portable in Section 5, is listed as future work (Section 10) rather than claimed here.

9 REPRODUCIBILITY AND ARTIFACT AVAILABILITY

Every number in this paper traces back to a named, re-runnable script and a checked-in CSV or log in this project's repository, organized so results from different sessions and hardware can be compared later without re-deriving them or trusting memory: `revalidation/correctness/` holds the Section 4 bug evidence; `revalidation/benchmarks/` and `revalidation/harness/` hold the cross-machine wall-clock and hardware-counter data behind Section 7; `src/avx2/ladder/` holds the Section 5 measurement- bias harness and its raw sweep/pinning-experiment CSVs; and `revalidation/dumas2024/` holds the Section 6 formula-verification script, correctness sweep, and accuracy/performance comparison data. Each directory's own `README.md` documents exact machine/compiler/command-line provenance for its data. `REPRODUCIBILITY.md` is the single top-level entry point tying all of the above together end-to-end (pinned to commit `2b78eaa`), and the repository is released under the MIT license (`LICENSE`), with the curated OpenBLAS/BLIS excerpts in `src/reference/` kept under their own original upstream BSD-3-Clause terms. A clone-and-run harness (`reproduce.sh` for Linux/macOS/ WSL, `reproduce.ps1` for native Windows) builds and runs the portable subset of this paper's claims – classical-DGEMM correctness and performance, both Strassen rebuilds, and the accuracy sweep – in a couple of minutes on any AVX2+FMA+OpenMP machine, writing a machine-identified CSV row (OS, CPU, compiler, core counts, correctness pass/fail, headline GFLOP/s) designed so results collected from several independent machines concatenate directly into one cross-machine comparison table. **[TODO: Remaining step, per FINDINGS.md Part L.3.2: tag this commit and connect the repository to Zenodo (or upload a tagged source archive directly) to mint a DOI, so the artifact is independently citable regardless of where this paper's prose is eventually published – this is an external-service, author-initiated step not done as part of drafting this paper.]** This submission targets ACM TOMS, pursuing the Replicated Computational Results track on top of standard review – a structural match for Section 4 specifically, since an RCR reviewer independently re-running this paper's own computational results is an independent, external check on every bug claimed there, regardless of the audited implementation's own publication status. An arXiv preprint is posted independently of this submission and does not substitute for it.

10 CONCLUSION AND FUTURE WORK

We set out to check whether an earlier, unpublished AVX-512 DGEMM/Strassen implementation does what it claims, whether its reported performance wins are real hardware effects, and whether a very recent (2024) result in the numerical-accuracy literature could be benchmarked for the first time using the same infrastructure. On the first question, we found and fixed four real correctness bugs (plus six more in a deeper second pass), the most serious silently transposing the product of a non-square multiplication. On the second, a large scheduling-policy performance win did not survive proper repeated measurement, while raw core-placement effects did – a concrete, domain-specific demonstration that measurement bias (Mytkowicz et al., 2009) remains a live concern in exactly the kind of virtualized development environment most GEMM tuning work is now done in. On the third, we implemented and measured Dumas et al.'s 2024 alternate-basis Strassen variant

for what appears to be the first time under realistic execution, confirming its accuracy claim and quantifying, for the first time, the performance cost that comes with it.

Concrete next steps, in priority order: cross-machine confirmation of Table 4's performance-cost figure on `glogin` and native Windows; project-wide adoption of Section 3.2's bootstrap-CI protocol beyond Section 5; and running the measurement-bias protocol from Section 5 *inside* WSL2/Hyper-V itself (not just outside it), to settle directly whether the original scheduling win was virtualization-compensation rather than merely unreproduced elsewhere. One further direction, noted but deliberately not built out here: the same statistical/hardware-counter measurement rigor this paper applies throughout is largely absent from the fast- growing LLM-driven kernel-generation literature (Section 2), which is presently evaluated mostly on single or best-of-N wall-clock numbers; using this project's harness to evaluate an LLM-generated kernel under the same protocol would be a natural, low-risk extension, left for future work rather than claimed as a contribution of this paper.

ACKNOWLEDGMENTS

We thank the Virginia Tech Department of Computer Science for access to the `glogin.cs.vt.edu` compute cluster used throughout Sections 7 and 6's hardware-counter, cross-compiler, and accuracy/performance measurements. **[TODO: Add any additional funding, advisor, or reviewer acknowledgments here.]**

REFERENCES

[1] 2015. Editorial: ACM TOMS Replicated Computational Results Initiative. *ACM Trans. Math. Software* 41, 3 (2015).
[2] 2015. GEMMbench: A Framework for Reproducible and Collaborative Benchmarking of Matrix Multiplication. arXiv:1511.03742.
[3] 2022. Research Artifacts and Citations in Computer Systems Papers. *PeerJ Computer Science* (2022). PMC9044204.
[4] 2025. QiMeng-GEMM: Automatically Generating High-Performance Matrix Multiplication Code. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
[5] 2025. Report on Challenges of Practical Reproducibility for Systems and HPC Computer Science. arXiv:2505.01671.
[6] 2026. Towards Automated Kernel Generation in the Era of LLMs. arXiv:2601.15727.
[7] Grey Ballard, Austin R. Benson, Alex Druinsky, Benjamin Lipshitz, and Oded Schwartz. 2016. Improving the Numerical Stability of Fast Matrix Multiplication. *SIAM J. Matrix Anal. Appl.* 37, 4 (2016), 1382–1418. https://doi.org/10.1137/15M1032168
[8] Grey Ballard, James Demmel, Olga Holtz, and Oded Schwartz. 2012. Communication-Optimal Parallel Algorithm for Strassen's Matrix Multiplication. In *Proceedings of the 24th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA'12)*. https://doi.org/10.1145/2312005.2312044 arXiv:1202.3173.
[9] Ethan Davis. 2025. High Performance Matrix Multiplication. arXiv:2509.04594.
[10] J. J. Dongarra, Jeremy Du Croz, Iain S. Duff, and Sven Hammarling. 1990. A Set of Level 3 Basic Linear Algebra Subprograms. *ACM Trans. Math. Software* 16, 1 (1990), 1–17. https://doi.org/10.1145/77626.79170
[11] Jean-Guillaume Dumas, Clément Pernet, and Alexandre Sedoglavic. 2024. Strassen's Algorithm Is Not Optimally Accurate. In *Proceedings of the 2024 International Symposium on Symbolic and Algebraic Computation (ISSAC'24)*. https://doi.org/10.1145/3666000.3669697 arXiv:2402.05630.
[12] Andy Georges, Dries Buytaert, and Lieven Eeckhout. 2007. Statistically Rigorous Java Performance Evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'07)*. 57–76. https://doi.org/10.1145/1297027.1297033
[13] Kazushige Goto and Robert A. van de Geijn. 2008. Anatomy of High-Performance Matrix Multiplication. *ACM Trans. Math. Software* 34, 3, Article 12 (2008). https://doi.org/10.1145/1356052.1356053
[14] Nicholas J. Higham. 1990. Exploiting Fast Matrix Multiplication Within the Level 3 BLAS. *ACM Trans. Math. Software* 16, 4 (1990), 352–368. https://doi.org/10.1145/98267.98290
[15] Jianyu Huang, Tyler M. Smith, Greg M. Henry, and Robert A. van de Geijn. 2016. Strassen's Algorithm Reloaded. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'16)*. https://doi.org/10.5555/3014904.3014983
[16] Jianyu Huang, Chenhan D. Yu, and Robert A. van de Geijn. 2020. Strassen's Algorithm Reloaded on GPUs. *ACM Trans. Math. Software* 46, 1, Article 3 (2020). https://doi.org/10.1145/3372419

[17] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. 2009. Producing Wrong Data Without Doing Anything Obviously Wrong!. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'09)*. https://doi.org/10.1145/1508244.1508275

[18] Volker Strassen. 1969. Gaussian Elimination is Not Optimal. *Numer. Math.* 13, 4 (1969), 354–356. https://doi.org/10.1007/BF02165411

[19] Field G. Van Zee and Robert A. van de Geijn. 2015. BLIS: A Framework for Rapidly Instantiating BLAS Functionality. *ACM Trans. Math. Software* 41, 3, Article 14 (2015). https://doi.org/10.1145/2764454

[20] Vincent M. Weaver. 2013. Linux perf_event Features and Overhead. In *Proceedings of the 2nd International Workshop on Performance Analysis of Workload Optimized Systems (FastPath)*.

[21] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (2009), 65–76. https://doi.org/10.1145/1498765.1498785